



Zoho Schools for Graduate Studies



Notes

Modeling the 'Car' Object

Attributes (Instance Variables):

- `String color`: To store the color of the car.
- `float mileage`: To store the car's mileage (kilometers per liter).

Behavior (Method):

- `void start()`: A method to represent the car starting, which prints "Car is started...".

```
Pen.java Car.java X
1 package part2;
2
3 public class Car {
4     String color;
5     float mileage;
6
7     void start() {
8         System.out.println("Car started...");
9     }
10 }
11
```

The Problem with Direct Access

- A `CarDemo` class was created to instantiate and use the `Car` object.

Initial Code Problem: The attributes (`color`, `mileage`) were directly accessed from the `CarDemo` class (e.g., `c.color`).

```
// In CarDemo.java
public class CarDemo {
    public static void main(String[] args) {
        Car c = new Car();
        System.out.println(c.color);
        System.out.println(c.mileage);
        c.start();
    }
}
```

Output:

null

0.0

Car started...

- Key Issue Identified:

This is a "bad programming practice." One class's members should not be directly accessible by another class. This can lead to uncontrolled modifications (e.g., "anyone can modify a member directly").

Introducing Encapsulation: The `private` Keyword

To prevent direct access, the `color` and `mileage` attributes in the `Car` class were declared as `private`.

```
// In Car.java
public class Car {
    private String color;
    private float mileage;
    // ... methods
}
```

- A `private` member is only accessible within its own class.
- This change caused a compilation error in `CarDemo` because `c.color` was no longer visible.

Indirect Access: Getters and Setters

- Direct access is prevented, but we still need a controlled way to read and modify the private attributes. This is achieved through public methods.

Getter (Accessor Method)

- A method designed to read the value of a private attribute.

Example: `public String getColor()` returns the value of the `color` variable.

```
// In Car.java
// getter or accessor method

String getColor() {
    return color;
}
```

Setter (Mutator/Modifier Method)

- A method designed to **modify** the value of a private attribute.

Example: `public void setColor(String color)` takes a new color as an argument and assigns it to the private `color` variable.

// In Car.java

// setter or modifier or mutator method

```
public void setColor(String c) {  
    color = c;  
}
```

Car.java (Encapsulated)

```
public class Car {  
    private String color;  
    private float mileage;  
  
    // Getter for color  
    public String getColor() {  
        return color;  
    }  
  
    // Setter for color  
    public void setColor(String newColor) {  
        color = newColor;  
    }  
  
    // ... other methods  
}
```

The `this` Keyword

- A problem arises if the parameter name in a setter is the same as the instance variable name (e.g., `setColor(String color)` where the instance variable is also `color`).

The line `color = color;` becomes ambiguous. The compiler prioritizes the local variable.

- **Solution:**

Use the `this` keyword. `this` is a reference to the currently invoked object.

Correct Implementation: `this.color = color;` explicitly states that the instance variable (`this.color`) should be assigned the value of the parameter (`color`).

Ambiguous Code (Incorrect)

```
public void setColor(String color) {  
    color = color;  
    // Assigns parameter to itself!  
}
```

Clear Code with 'this' (Correct)

```
public void setColor(String color) {  
    this.color = color;  
    // Instance var = parameter  
}
```

Encapsulation

- **Definition:** The process of binding (or bundling) attributes and the methods that operate on them into a single logical unit, the class.
- Implementing a class itself is an act of encapsulation.
- **Fully Encapsulated (or Strictly Encapsulated) Class:** A class where all instance variables are **private** and all methods are **public**. This is the ideal to aim for.

Coupling

- **Definition:** The degree of dependency one object has on another.
- **Goal:** Aim for **Loose Coupling** (or Weak Coupling). Objects should be as independent as possible, with minimal reliance on the internal workings of other objects.

Cohesion

- **Definition:** The degree to which the elements inside a single module (or class) belong together. It's a measure of self-dependency.
- **Goal:** Aim for **High Cohesion**. A class should be focused on a single, well-defined purpose.

Summary of Best Practices

1. Mark instance variables/attributes as **private**.
2. Provide **public** getter and setter methods for controlled access to private attributes.
3. Design objects to be **loosely coupled** and **highly cohesive**.
4. An "object reference" (like **c** in **Car c = new Car();**) is not a direct memory address but a unique, system-assigned alphanumeric string used to identify the object.